

Làm sạch và chuẩn bị dữ liệu

Contents

- 8.1. Xử lý dữ liệu không quan sát được
- 8.2. Chuyển đổi Dữ Liệu
- 8.3. Thao tác trên chuỗi ký tự
- 8.4. Dữ liệu kiểu phân loại



KHOA HỌC DỮ LIỆU TRONG KINH TẾ VÀ KINH DOANH

Trong quá trình phân tích và xây dựng mô hình dữ liệu, một phần đáng kể thời gian thường được dành cho công việc chuẩn bị dữ liệu, bao gồm: thu thập, làm sạch, chuyển đổi và cấu trúc lại dữ liệu. Nhiều nghiên cứu thực nghiệm đã ghi nhận rằng các tác vụ này có thể chiếm tới 80% hoặc hơn tổng thời gian làm việc của một nhà phân tích dữ liệu.

Một trong những thách thức phổ biến là dữ liệu được lưu trữ dưới các định dạng không phù hợp với yêu cầu của bài toán phân tích cụ thể. Do đó, các nhà phân tích thường phải thực hiện các thao tác chuyển đổi linh hoạt giữa các cấu trúc dữ liệu, sử dụng những ngôn ngữ lập trình như Python, R, hoặc Java, cũng như các công cụ dòng lệnh truyền thống trong môi trường Unix.

Tuy nhiên, sự kết hợp giữa thư viện `pandas` và các khả năng tích hợp sẵn của ngôn ngữ Python đã cung cấp một bộ công cụ thao tác dữ liệu có hiệu quả cao, có thể mở rộng, giúp giải quyết nhanh chóng hầu hết các nhu cầu xử lý dữ liệu phổ biến trong thực tiễn nghiên cứu và ứng dụng.

Trong trường hợp người dùng phát hiện một thao tác xử lý dữ liệu chưa được đề cập trong tài liệu này hoặc chưa được hỗ trợ trong pandas, việc chia sẻ trường hợp sử dụng cụ thể lên cộng đồng phát triển pandas trên GitHub hoặc các danh sách thư tín Python là rất được khuyến khích.

Trên thực tế, nhiều cải tiến trong thiết kế và triển khai pandas đã trực tiếp xuất phát từ các yêu cầu thực tế như vậy.

Chương này sẽ tập trung trình bày các công cụ hỗ trợ xử lý dữ liệu thiếu, dữ liệu trùng lặp, thao tác trên chuỗi ký tự, cùng với một số phép biến đổi dữ liệu phục vụ mục tiêu phân tích. Trong chương kế tiếp, chúng ta sẽ tiếp tục tìm hiểu về các kỹ thuật kết hợp và tái tổ chức tập dữ liệu, qua đó hỗ trợ cho quá trình phân tích và trực quan hóa dữ liệu một cách hiệu quả hơn.

```
import numpy as np
import pandas as pd
PREVIOUS_MAX_ROWS = pd.options.display.max_rows
pd.options.display.max_rows = 20
np.random.seed(12345) # để ví dụ có thể lặp lại
import matplotlib.pyplot as plt
plt.rc("figure", figsize=(10, 6))
np.set_printoptions(precision=4, suppress=True)
```

8.1. Xử lý dữ liệu không quan sát được

Dữ liệu không quan sát được (missing data) thường xuyên xuất hiện trong nhiều ứng dụng phân tích dữ liệu. Một trong những mục tiêu của `pandas` là làm cho việc xử lý dữ liệu không quan sát được trở nên dễ dàng nhất có thể. Ví dụ, tất cả các thống kê mô tả trên các đối tượng của `pandas` đều loại trừ dữ liệu này theo mặc định.

Cách biểu diễn dữ liệu không quan sát được trong các đối tượng `pandas` còn nhiều vấn đề, nhưng cũng đủ cho hầu hết các mục đích sử dụng trong thực tế. Đối với dữ liệu có kiểu `float64`, pandas sử dụng giá trị dấu phẩy động `NaN` (Not a Number) để biểu diễn dữ liệu bị thiếu.

Chúng ta gọi đây là một *giá trị sentinel*: khi có mặt, nó chỉ ra một giá trị bị không quan sát được (hoặc null):

```
float_data = pd.Series([1.2, -3.5, np.nan, 0])
float_data
# Out[15]:
# 0    1.2
# 1   -3.5
# 2    NaN
# 3    0.0
# dtype: float64
```

```
0    1.2
1   -3.5
2    NaN
3    0.0
dtype: float64
```

Phương thức `isna` trả về một mảng kiểu logical với giá trị `True` ở những nơi giá trị là không quan sát được:

```
float_data.isna()
# Out[16]:
# 0    False
# 1    False
# 2     True
# 3    False
# dtype: bool
```

```
0    False
1    False
2     True
3    False
dtype: bool
```

Trong pandas cũng áp dụng một quy ước được sử dụng trong ngôn ngữ lập trình R bằng cách gọi dữ liệu không quan sát được là `NA`, viết tắt của *not available*. Trong các ứng dụng thống kê, dữ liệu `NA` có thể là dữ liệu không tồn tại hoặc tồn tại nhưng không được quan sát (ví dụ, do các vấn đề trong quá trình thu thập dữ liệu). Khi dọn dẹp dữ liệu để phân tích, việc phân tích chính dữ liệu `NA` thường rất quan trọng để xác định các vấn đề thu thập dữ liệu hoặc các sai lệch tiềm ẩn trong dữ liệu do dữ liệu bị thiếu gây ra.

Giá trị `None` tích hợp sẵn của Python cũng được coi là `NA`:

```

string_data = pd.Series(["aardvark", np.nan, None, "avocado"])
string_data
# Out[18]:
# 0    aardvark
# 1         NaN
# 2         None
# 3    avocado
# dtype: object

string_data.isna()
# Out[19]:
# 0    False
# 1     True
# 2     True
# 3    False
# dtype: bool

```

```

0    False
1     True
2     True
3    False
dtype: bool

```

```

float_data = pd.Series([1, 2, None], dtype='float64')
float_data
# Out[21]:
# 0    1.0
# 1    2.0
# 2    NaN
# dtype: float64

float_data.isna()
# Out[22]:
# 0    False
# 1    False
# 2     True
# dtype: bool

```

```

0    False
1    False
2     True
dtype: bool

```

Thư viện `pandas` đã cố gắng làm cho việc xử lý dữ liệu không quan sát được nhất quán giữa các kiểu dữ liệu. Xem Bảng ... để biết danh sách một số hàm liên quan đến xử lý dữ liệu không quan sát được.

Bảng ...: Các phương thức xử lý NA

Phương thức	Mô tả
<code>dropna</code>	Lọc các chỉ số dựa trên việc các giá trị cho chỉ số có dữ liệu bị thiếu hay không
<code>fillna</code>	Điền vào dữ liệu <code>NA</code> bằng một giá trị nào đó hoặc sử dụng một phương pháp nội suy như <code>"ffill"</code> hoặc <code>"bfill"</code> .
<code>isna</code>	Trả về các giá trị boolean cho biết giá trị nào là <code>NA</code> .
<code>notna</code>	Phủ định của <code>isna</code> , trả về <code>True</code> cho các giá trị không phải NA và <code>False</code> cho các giá trị NA.

8.1.1. Lọc bỏ dữ liệu bị thiếu

Có nhiều cách để lọc bỏ dữ liệu bị thiếu. Mặc dù bạn luôn có thể tự làm điều đó bằng cách sử dụng `pandas.notna` và lọc dữ liệu bằng chỉ số kiểu logical. Ngoài ra bạn có thể dùng phương thức `dropna` như đã mô tả ở trên:

```
data = pd.Series([1, np.nan, 3.5, np.nan, 7])
data.dropna()
# Out[24]:
# 0    1.0
# 2    3.5
# 4    7.0
# dtype: float64
```

```
0    1.0
2    3.5
4    7.0
dtype: float64
```

Điều này tương đương với:

```
data[data.notna()]
# Out[25]:
# 0    1.0
# 2    3.5
# 4    7.0
# dtype: float64
```

```
0    1.0
2    3.5
4    7.0
dtype: float64
```

Với các đối tượng DataFrame, có nhiều cách khác nhau để loại bỏ dữ liệu không quan sát được. Bạn có thể muốn loại bỏ các hàng hoặc cột hoàn toàn là **NA**, hoặc chỉ những hàng hoặc cột chứa bất kỳ **NA** nào. **dropna** theo mặc định sẽ loại bỏ bất kỳ hàng nào chứa một giá trị không quan sát được:

```
data = pd.DataFrame([[1., 6.5, 3.], [1., np.nan, np.nan],
                    [np.nan, np.nan, np.nan], [np.nan, 6.5, 3.]])
data
# Out[27]:
#      0      1      2
# 0  1.0  6.5  3.0
# 1  1.0  NaN  NaN
# 2  NaN  NaN  NaN
# 3  NaN  6.5  3.0

data.dropna()
# Out[28]:
#      0      1      2
# 0  1.0  6.5  3.0
```

	0	1	2
0	1.0	6.5	3.0

Sử dụng tham số **how="all"** sẽ chỉ loại bỏ các hàng mà tất cả các giá trị đều là **NA**:

```
data.dropna(how="all")
# Out[29]:
#      0      1      2
# 0  1.0  6.5  3.0
# 1  1.0  NaN  NaN
# 3  NaN  6.5  3.0
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
3	NaN	6.5	3.0

Hãy lưu ý rằng các hàm này trả về các đối tượng mới theo mặc định và không sửa đổi nội dung của đối tượng ban đầu.

Để loại bỏ các cột theo cách tương tự, hãy sử dụng tham số `axis="columns"`:

```
data[4] = np.nan
data
# Out[31]:
#      0      1      2      4
# 0  1.0  6.5  3.0  NaN
# 1  1.0  NaN  NaN  NaN
# 2  NaN  NaN  NaN  NaN
# 3  NaN  6.5  3.0  NaN

data.dropna(axis="columns", how="all")
# Out[32]:
#      0      1      2
# 0  1.0  6.5  3.0
# 1  1.0  NaN  NaN
# 2  NaN  NaN  NaN
# 3  NaN  6.5  3.0
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

Giả sử bạn muốn giữ lại chỉ những hàng chứa tối đa một số lượng `NA` nhất định. Bạn có thể chỉ ra điều này bằng tham số `thresh`:

```

df = pd.DataFrame(np.random.standard_normal((7, 3)))
df.iloc[:4, 1] = np.nan
df.iloc[:2, 2] = np.nan
df
# Out[36]:
#           0           1           2
# 0 -0.204708      NaN      NaN
# 1 -0.555730      NaN      NaN
# 2  0.092908      NaN  0.769023
# 3  1.246435      NaN -1.296221
# 4  0.274992  0.228913  1.352917
# 5  0.886429 -2.001637 -0.371843
# 6  1.669025 -0.438570 -0.539741

df.dropna()
# Out[37]:
#           0           1           2
# 4  0.274992  0.228913  1.352917
# 5  0.886429 -2.001637 -0.371843
# 6  1.669025 -0.438570 -0.539741

df.dropna(thresh=2)
# Out[38]:
#           0           1           2
# 2  0.092908      NaN  0.769023
# 3  1.246435      NaN -1.296221
# 4  0.274992  0.228913  1.352917
# 5  0.886429 -2.001637 -0.371843
# 6  1.669025 -0.438570 -0.539741

```

	0	1	2
2	0.092908	NaN	0.769023
3	1.246435	NaN	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741

8.1.2. Điền vào dữ liệu bị thiếu

Thay vì lọc bỏ dữ liệu không quan sát được và có khả năng loại bỏ dữ liệu khác cùng hàng hoặc cùng cột, bạn có giải pháp là điền vào các giá trị này theo nhiều cách. Đối với hầu hết các mục đích, phương thức `fillna` là hàm chính được sử dụng. Phương pháp đơn giản nhất là gọi `fillna` với một hằng số để thay thế các giá trị `NA` bằng giá trị đó:


```
df.fillna(0)
# Out[39]:
#           0           1           2
# 0 -0.204708  0.000000  0.000000
# 1 -0.555730  0.000000  0.000000
# 2  0.092908  0.000000  0.769023
# 3  1.246435  0.000000 -1.296221
# 4  0.274992  0.228913  1.352917
# 5  0.886429 -2.001637 -0.371843
# 6  1.669025 -0.438570 -0.539741
```

	0	1	2
0	-0.204708	0.000000	0.000000
1	-0.555730	0.000000	0.000000
2	0.092908	0.000000	0.769023
3	1.246435	0.000000	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741

Gọi `fillna` với tham số là một dictionary, bạn có thể điền các giá trị khác nhau cho mỗi cột:

```
df.fillna({1: 0.5, 2: 0})
# Out[40]:
#           0           1           2
# 0 -0.204708  0.500000  0.000000
# 1 -0.555730  0.500000  0.000000
# 2  0.092908  0.500000  0.769023
# 3  1.246435  0.500000 -1.296221
# 4  0.274992  0.228913  1.352917
# 5  0.886429 -2.001637 -0.371843
# 6  1.669025 -0.438570 -0.539741
```

	0	1	2
0	-0.204708	0.500000	0.000000
1	-0.555730	0.500000	0.000000
2	0.092908	0.500000	0.769023
3	1.246435	0.500000	-1.296221
4	0.274992	0.228913	1.352917
5	0.886429	-2.001637	-0.371843
6	1.669025	-0.438570	-0.539741

Các phương pháp nội suy tương tự có sẵn cho việc sắp xếp lại (xem Bảng 5.3) có thể được sử dụng với `fillna`:

```
df = pd.DataFrame(np.random.standard_normal((6, 3)))
df.iloc[2:, 1] = np.nan
df.iloc[4:, 2] = np.nan
df
# Out[44]:
#           0           1           2
# 0  0.476985  3.248944 -1.021228
# 1 -0.577087  0.124121  0.302614
# 2  0.523772      NaN  1.343810
# 3 -0.713544      NaN -2.370232
# 4 -1.860761      NaN      NaN
# 5 -1.265934      NaN      NaN

df.fillna(method="ffill")
# Out[45]:
#           0           1           2
# 0  0.476985  3.248944 -1.021228
# 1 -0.577087  0.124121  0.302614
# 2  0.523772  0.124121  1.343810
# 3 -0.713544  0.124121 -2.370232
# 4 -1.860761  0.124121 -2.370232
# 5 -1.265934  0.124121 -2.370232

df.fillna(method="ffill", limit=2)
# Out[46]:
#           0           1           2
# 0  0.476985  3.248944 -1.021228
# 1 -0.577087  0.124121  0.302614
# 2  0.523772  0.124121  1.343810
# 3 -0.713544  0.124121 -2.370232
# 4 -1.860761      NaN -2.370232
# 5 -1.265934      NaN -2.370232
```

	0	1	2
0	0.476985	3.248944	-1.021228
1	-0.577087	0.124121	0.302614
2	0.523772	0.124121	1.343810
3	-0.713544	0.124121	-2.370232
4	-1.860761	NaN	-2.370232
5	-1.265934	NaN	-2.370232

Với `fillna`, bạn có thể làm điền dữ liệu bằng các giá trị thống kê đặc trưng như trung vị hoặc trung bình:

```
data = pd.Series([1., np.nan, 3.5, np.nan, 7])
data.fillna(data.mean())
# Out[48]:
# 0    1.000000
# 1    3.833333
# 2    3.500000
# 3    3.833333
# 4    7.000000
# dtype: float64
```

```
0    1.000000
1    3.833333
2    3.500000
3    3.833333
4    7.000000
dtype: float64
```

Xem Bảng ... để tham khảo về các tham số của hàm `fillna`.

Bảng...: các tham số của hàm `fillna`

Tham số	Mô tả
<code>value</code>	Giá trị vô hướng hoặc đối tượng giống dictionary được sử dụng để điền vào các giá trị bị thiếu.
<code>method</code>	Phương pháp nội suy; một trong <code>"bfill"</code> (điền lùi) hoặc <code>"ffill"</code> (điền tiến); mặc định là <code>None</code> .
<code>axis</code>	Trục cần điền (<code>"index"</code> hoặc <code>"columns"</code>); mặc định là <code>axis="index"</code> .
<code>limit</code>	Đối với điền tiến và lùi, số lượng tối đa các khoảng trống liên tiếp cần điền.

8.2. Chuyển đổi Dữ Liệu

Cho đến nay trong chương này, chúng ta đã thảo luận về xử lý dữ liệu bị thiếu. Ngoài ra, lọc dữ liệu và làm sạch làm sạch dữ liệu cũng là các bước xử lý quan trọng khác.

8.2.1. Loại bỏ các giá trị trùng lặp

Các hàng trùng lặp có thể được tìm thấy trong một DataFrame vì nhiều lý do. Đây là một ví dụ:

```
data = pd.DataFrame({"k1": ["one", "two"] * 3 + ["two"],
                     "k2": [1, 1, 2, 3, 3, 4, 4]})

data
# Out[50]:
#      k1  k2
# 0  one   1
# 1  two   1
# 2  one   2
# 3  two   3
# 4  one   3
# 5  two   4
# 6  two   4
```

	k1	k2
0	one	1
1	two	1
2	one	2
3	two	3
4	one	3
5	two	4
6	two	4

Phương thức `uplicated` của DataFrame trả về một mảng kiểu logical cho biết liệu mỗi hàng có phải là một bản sao, nghĩa là các giá trị của nó hoàn toàn bằng với các giá trị trong một hàng trước đó, hay không:

```
data.duplicated()
# Out[51]:
# 0    False
# 1    False
# 2    False
# 3    False
# 4    False
# 5    False
# 6     True
# dtype: bool
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6     True
dtype: bool
```

Phương thức `drop_duplicates` trả về một DataFrame với các hàng mà mảng `uplicated` là `False` được lọc bỏ:

```
data.drop_duplicates()
# Out[52]:
#      k1  k2
# 0  one   1
# 1  two   1
# 2  one   2
# 3  two   3
# 4  one   3
# 5  two   4
```

	k1	k2
0	one	1
1	two	1
2	one	2
3	two	3
4	one	3
5	two	4

Cả hai phương thức này theo mặc định xem xét tất cả các cột. Tuy nhiên, bạn cũng có thể chỉ định bất kỳ tập hợp con nào của các cột để phát hiện ra các giá trị bị trùng lặp. Giả sử chúng ta có một cột giá trị bổ sung và muốn lọc các bản sao chỉ dựa trên cột `"k1"`:

```
data["v1"] = range(7)
data
# Out[54]:
#      k1  k2  v1
# 0  one   1   0
# 1  two   1   1
# 2  one   2   2
# 3  two   3   3
# 4  one   3   4
# 5  two   4   5
# 6  two   4   6

data.drop_duplicates(subset=["k1"])
# Out[55]:
#      k1  k2  v1
# 0  one   1   0
# 1  two   1   1
```

	k1	k2	v1
0	one	1	0
1	two	1	1

`uplicated` và `drop_duplicates` theo mặc định giữ lại tổ hợp giá trị được quan sát đầu tiên. Sử dụng tham số `keep="last"` sẽ loại bỏ các hàng bị lặp xuất hiện ở phía trên:

```
data.drop_duplicates(["k1", "k2"], keep="last")
# Out[56]:
#      k1  k2  v1
# 0  one   1   0
# 1  two   1   1
# 2  one   2   2
# 3  two   3   3
# 4  one   3   4
# 6  two   4   6
```

	k1	k2	v1
0	one	1	0
1	two	1	1
2	one	2	2
3	two	3	3
4	one	3	4
6	two	4	6

8.2.2. Chuyển đổi dữ liệu bằng hàm hoặc mapping

Đối với nhiều tập dữ liệu, bạn có thể muốn thực hiện một số chuyển đổi dựa trên các giá trị trong một mảng, Series hoặc cột trong DataFrame. Hãy xem xét dữ liệu giả định sau được thu thập về các loại thực phẩm khác nhau:

```
data = pd.DataFrame({"food": ["bacon", "pulled pork", "bacon",
                             "pastrami", "corned beef", "bacon",
                             "pastrami", "honey ham", "nova lox"],
                    "ounces": [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})
```

```
data
# Out[58]:
#      food  ounces
# 0    bacon     4.0
# 1 pulled pork     3.0
# 2    bacon    12.0
# 3  pastrami     6.0
# 4  corned beef     7.5
# 5    bacon     8.0
# 6  pastrami     3.0
# 7  honey ham     5.0
# 8   nova lox     6.0
```

	food	ounces
0	bacon	4.0
1	pulled pork	3.0
2	bacon	12.0
3	pastrami	6.0
4	corned beef	7.5
5	bacon	8.0
6	pastrami	3.0
7	honey ham	5.0
8	nova lox	6.0

Giả sử bạn muốn thêm một cột cho biết loại động vật mà mỗi loại thực phẩm đến từ đó. Cần có một mapping của từng loại thịt riêng biệt đến loại động vật:

```
meat_to_animal = {
    "bacon": "pig",
    "pulled pork": "pig",
    "pastrami": "cow",
    "corned beef": "cow",
    "honey ham": "pig",
    "nova lox": "salmon"
}
```


Phương thức `map` trên một mảng được chấp nhận như một hàm để thực hiện việc thêm một cột mới `animal` vào dữ liệu

```
data["animal"] = data["food"].map(meat_to_animal)
data
# Out[61]:
#      food  ounces  animal
# 0    bacon    4.0    pig
# 1 pulled pork    3.0    pig
# 2    bacon   12.0    pig
# 3  pastrami    6.0    cow
# 4 corned beef    7.5    cow
# 5    bacon    8.0    pig
# 6  pastrami    3.0    cow
# 7 honey ham    5.0    pig
# 8   nova lox    6.0  salmon
```

	food	ounces	animal
0	bacon	4.0	pig
1	pulled pork	3.0	pig
2	bacon	12.0	pig
3	pastrami	6.0	cow
4	corned beef	7.5	cow
5	bacon	8.0	pig
6	pastrami	3.0	cow
7	honey ham	5.0	pig
8	nova lox	6.0	salmon

Hoặc chúng ta cũng có thể định nghĩa một hàm thay cho phương thức `map`

```
def get_animal(x):  
    return meat_to_animal[x]  
  
data["food"].map(get_animal)  
# Out[63]:  
# 0      pig  
# 1      pig  
# 2      pig  
# 3      cow  
# 4      cow  
# 5      pig  
# 6      cow  
# 7      pig  
# 8      salmon  
# Name: food, dtype: object
```

```
0      pig  
1      pig  
2      pig  
3      cow  
4      cow  
5      pig  
6      cow  
7      pig  
8      salmon  
Name: food, dtype: object
```

Nhìn chung, sử dụng `map` là một cách thuận tiện để thực hiện các phép biến đổi theo từng phần tử và các hoạt động liên quan đến làm sạch dữ liệu khác.

8.2.3. Thay thế các giá trị

Điền vào dữ liệu bị thiếu bằng phương thức `fillna` là một trường hợp đặc biệt của việc thay thế giá trị trong trường hợp tổng quát hơn. Như chúng ta đã thấy, `map` có thể được sử dụng để sửa đổi một tập hợp con các giá trị trong một đối tượng, nhưng `replace` cung cấp một cách đơn giản và linh hoạt hơn để làm điều đó. Hãy xem xét Series này:

```
data = pd.Series([1., -999., 2., -999., -1000., 3.])
data
# Out[65]:
# 0      1.0
# 1    -999.0
# 2      2.0
# 3    -999.0
# 4   -1000.0
# 5      3.0
# dtype: float64
```

```
0      1.0
1    -999.0
2      2.0
3    -999.0
4   -1000.0
5      3.0
dtype: float64
```

Giá trị `-999` có thể là giá trị sentinel cho dữ liệu không quan sát được. Để thay thế những giá trị này bằng các giá trị `NA` mà pandas hiểu, chúng ta có thể sử dụng `replace`, tạo ra một Series mới:

```
data.replace(-999, np.nan)
# Out[66]:
# 0      1.0
# 1      NaN
# 2      2.0
# 3      NaN
# 4   -1000.0
# 5      3.0
# dtype: float64
```

```
0      1.0
1      NaN
2      2.0
3      NaN
4   -1000.0
5      3.0
dtype: float64
```

Nếu bạn muốn thay thế nhiều giá trị cùng một lúc, bạn thay vào một danh sách và sau đó là giá trị thay thế:

```
data.replace([-999, -1000], np.nan)
# Out[67]:
# 0    1.0
# 1    NaN
# 2    2.0
# 3    NaN
# 4    NaN
# 5    3.0
# dtype: float64
```

```
0    1.0
1    NaN
2    2.0
3    NaN
4    NaN
5    3.0
dtype: float64
```

Để sử dụng một giá trị thay thế khác nhau cho mỗi giá trị, hãy sử dụng một danh sách các giá trị thay thế:

```
data.replace([-999, -1000], [np.nan, 0])
# Out[68]:
# 0    1.0
# 1    NaN
# 2    2.0
# 3    NaN
# 4    0.0
# 5    3.0
# dtype: float64
```

```
0    1.0
1    NaN
2    2.0
3    NaN
4    0.0
5    3.0
dtype: float64
```

Lưu ý: Phương thức `data.replace` khác với `data.str.replace`, phương thức này thực hiện thay thế chuỗi theo từng phần tử. Chúng ta sẽ xem xét các phương thức chuỗi này trên Series sau trong chương.

8.2.4. Đổi tên chỉ số hàng và cột

Giống như các giá trị trong một `Series`, các tên hàng và tên cột trong một `DataFrame` có thể được chuyển đổi tương tự bằng một hàm hoặc mapping để tạo ra các chỉ số mới. Bạn cũng có thể sửa đổi các chỉ số mà không cần tạo ra một cấu trúc dữ liệu mới. Đây là một ví dụ đơn giản:

```
data = pd.DataFrame(np.arange(12).reshape((3, 4)),
                    index=["Ohio", "Colorado", "New York"],
                    columns=["one", "two", "three", "four"])
data
```

	one	two	three	four
Ohio	0	1	2	3
Colorado	4	5	6	7
New York	8	9	10	11

Giống như một `Series`, chỉ số của trục có một phương thức `map`:

```
def transform(x):
    return x[:4].upper()

data.index.map(transform)
# Out[72]: Index(['OHIO', 'COLO', 'NEW'], dtype='object')
```

```
Index(['OHIO', 'COLO', 'NEW'], dtype='object')
```

Bạn có thể gán cho thuộc tính `index`, sửa đổi `DataFrame` trực tiếp:

```
data.index = data.index.map(transform)
data
# Out[74]:
#      one  two  three  four
# OHIO    0    1     2     3
# COLO    4    5     6     7
# NEW     8    9    10    11
```

	one	two	three	four
OHIO	0	1	2	3
COLO	4	5	6	7
NEW	8	9	10	11

Nếu bạn muốn tạo một phiên bản đã được chuyển đổi của một tập dữ liệu mà không sửa đổi bản gốc, một phương thức hữu ích là `rename`:

```
data.rename(index=str.title, columns=str.upper)
# Out[75]:
#      ONE  TWO  THREE  FOUR
# Ohio    0    1     2    3
# Colo    4    5     6    7
# New     8    9    10   11
```

	ONE	TWO	THREE	FOUR
Ohio	0	1	2	3
Colo	4	5	6	7
New	8	9	10	11

Đáng chú ý là `rename` có thể được sử dụng kết hợp với một đối tượng giống từ điển, cung cấp các giá trị mới cho một tập hợp con các tên hàng hoặc tên cột:

```
data.rename(index={"OHIO": "INDIANA"},
            columns={"three": "peekaboo"})
# Out[76]:
#      one  two  peekaboo  four
# INDIANA  0    1         2    3
# COLO     4    5         6    7
# NEW      8    9        10   11
```

	one	two	peekaboo	four
INDIANA	0	1	2	3
COLO	4	5	6	7
NEW	8	9	10	11

`rename` giúp bạn không phải thực hiện công việc sao chép DataFrame thủ công và gán các giá trị mới cho các thuộc tính `index` và `columns` của nó.

8.2.5. Rời rạc hóa và phân nhóm

Trong nhiều trường hợp, dữ liệu liên tục thường được rời rạc hóa hoặc tách thành các nhóm (bins) để phân tích. Giả sử bạn có dữ liệu về một nhóm người trong một nghiên cứu và bạn muốn nhóm họ thành các nhóm tuổi riêng biệt:

```
ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]
```

Yêu cầu đặt ra là chia những tuổi này thành các nhóm từ 18 đến 25, 26 đến 35, 36 đến 60 và cuối cùng là 61 tuổi trở lên. Để làm điều này, bạn phải sử dụng `pandas.cut`:

```
bins = [18, 25, 35, 60, 100]
age_categories = pd.cut(ages, bins)
age_categories
# Out[80]:
# [(18, 25], (18, 25], (18, 25], (25, 35], (18, 25], ..., (25, 35], (60, 100], (35, 60]
# Length: 12
# Categories (4, interval[int64, right]): [(18, 25] < (25, 35] < (35, 60] < (60, 100]]
```

```
[(18, 25], (18, 25], (18, 25], (25, 35], (18, 25], ..., (25, 35], (60, 100], (35, 60],
Length: 12
Categories (4, interval[int64, right]): [(18, 25] < (25, 35] < (35, 60] < (60, 100]]
```

Đối tượng mà `pandas` trả về là một đối tượng kiểu phân loại, hay categorical, đặc biệt. Kết quả hiển thị mô tả các nhóm được tính toán bởi `pandas.cut`. Mỗi nhóm được xác định bởi một kiểu giá trị khoảng đặc biệt chứa giới hạn dưới và giới hạn trên của mỗi nhóm:

```

age_categories.codes
# Out[81]: array([0, 0, 0, 1, 0, 0, 2, 1, 3, 2, 2, 1], dtype=int8)

age_categories.categories
# Out[82]: IntervalIndex([(18, 25], (25, 35], (35, 60], (60, 100]],
#                      dtype='interval[int64, right]')

age_categories.categories[0]
# Out[83]: Interval(18, 25, closed='right')

pd.value_counts(age_categories)
# Out[84]:
# (18, 25]      5
# (25, 35]      3
# (35, 60]      3
# (60, 100]     1
# Name: count, dtype: int64

```

```

(18, 25]      5
(25, 35]      3
(35, 60]      3
(60, 100]     1
Name: count, dtype: int64

```

Lưu ý rằng `pd.value_counts(categories)` là số lượng nhóm cho kết quả của `pandas.cut`.

Trong biểu diễn chuỗi của một khoảng, một dấu ngoặc đơn có nghĩa là phía đó mở (không bao gồm), trong khi dấu ngoặc vuông có nghĩa là phía đó đóng (bao gồm). Bạn có thể thay đổi phía nào được đóng bằng cách truyền `right=False`:

```

pd.cut(ages, bins, right=False)
# Out[85]:
# [[18, 25), [18, 25), [25, 35), [25, 35), [18, 25), ..., [25, 35), [60, 100), [35, 60)
# Length: 12
# Categories (4, interval[int64, left]): [[18, 25) < [25, 35) < [35, 60) < [60, 100)]

```

```

[[18, 25), [18, 25), [25, 35), [25, 35), [18, 25), ..., [25, 35), [60, 100), [35, 60),
Length: 12
Categories (4, interval[int64, left]): [[18, 25) < [25, 35) < [35, 60) < [60, 100)]

```

Bạn có thể ghi đè tên nhóm dựa trên khoảng mặc định bằng cách truyền một danh sách hoặc mảng cho tùy chọn `labels`:


```
group_names = ["Youth", "YoungAdult", "MiddleAged", "Senior"]
pd.cut(ages, bins, labels=group_names)
# Out[87]:
# ['Youth', 'Youth', 'Youth', 'YoungAdult', 'Youth', ..., 'YoungAdult', 'Senior', 'Mid
# Length: 12
# Categories (4, object): ['Youth' < 'YoungAdult' < 'MiddleAged' < 'Senior']
```

```
['Youth', 'Youth', 'Youth', 'YoungAdult', 'Youth', ..., 'YoungAdult', 'Senior', 'Middle
Length: 12
Categories (4, object): ['Youth' < 'YoungAdult' < 'MiddleAged' < 'Senior']
```

Nếu bạn gán một số nguyên, truyền tham số số lượng nhóm vào `pandas.cut` thay vì các giới hạn nhóm rõ ràng, `pandas` sẽ tính toán các nhóm có chiều dài bằng nhau dựa trên các giá trị tối thiểu và tối đa trong dữ liệu. Hãy xem xét trường hợp một số dữ liệu được phân bố đều được chia thành bốn phần:

```
data = np.random.uniform(size=20)
pd.cut(data, 4, precision=2)
# Out[89]:
# [(0.34, 0.55], (0.34, 0.55], (0.76, 0.97], (0.76, 0.97], (0.34, 0.55], ...,
# (0.34, 0.55], (0.34, 0.55], (0.55, 0.76], (0.34, 0.55], (0.12, 0.34]]
# Length: 20
# Categories (4, interval[float64, right]): [(0.12, 0.34] < (0.34, 0.55] < (0.55, 0.76]
```

```
[(0.34, 0.55], (0.34, 0.55], (0.76, 0.97], (0.76, 0.97], (0.34, 0.55], ..., (0.34, 0.55]
Length: 20
Categories (4, interval[float64, right]): [(0.12, 0.34] < (0.34, 0.55] < (0.55, 0.76] <
```

Tùy chọn `precision=2` giới hạn độ chính xác thập phân đến hai chữ số.

Một hàm liên quan chặt chẽ đến phân nhóm dữ liệu là `pandas.qcut` có thể phân nhóm dữ liệu dựa trên các phân vị. Tùy thuộc vào sự phân bố của dữ liệu, việc sử dụng `pandas.cut` thường sẽ không dẫn đến việc mỗi nhóm có cùng số lượng điểm dữ liệu. Vì `pandas.qcut` sử dụng các phân vị mẫu thay vào đó, bạn sẽ thu được các nhóm có kích thước gần bằng nhau:

```

data = np.random.standard_normal(1000)
quartiles = pd.qcut(data, 4, precision=2)
quartiles
# Out[92]:
# [(-0.026, 0.62], (0.62, 3.93], (-0.68, -0.026], (0.62, 3.93], (-0.026, 0.62], ...,
#  (-0.68, -0.026], (-0.68, -0.026], (-2.96, -0.68], (0.62, 3.93], (-0.68, -0.026]]
# Length: 1000
# Categories (4, interval[float64, right]): [(-2.96, -0.68] < (-0.68, -0.026] < (-0.02
pd.value_counts(quartiles)
# Out[93]:
# (-2.96, -0.68]      250
# (-0.68, -0.026]     250
# (-0.026, 0.62]      250
# (0.62, 3.93]        250
# Name: count, dtype: int64

```

```

(-2.96, -0.68]      250
(-0.68, -0.026]     250
(-0.026, 0.62]      250
(0.62, 3.93]        250
Name: count, dtype: int64

```

Tương tự như `pandas.cut`, bạn có thể tự tạo các phân vị theo ý mình mình:

```

pd.qcut(data, [0, 0.1, 0.5, 0.9, 1.]).value_counts()
# Out[94]:
# (-2.9499999999999997, -1.187]      100
# (-1.187, -0.0265]                  400
# (-0.0265, 1.286]                   400
# (1.286, 3.928]                      100
# Name: count, dtype: int64

```

```

(-2.9499999999999997, -1.187]      100
(-1.187, -0.0265]                  400
(-0.0265, 1.286]                   400
(1.286, 3.928]                      100
Name: count, dtype: int64

```

Chúng ta sẽ quay lại `pandas.cut` và `pandas.qcut` sau trong chương này trong phần thảo luận về tổng hợp và các thao tác nhóm, vì các hàm rời rạc hóa này đặc biệt hữu ích cho phân tích phân vị và nhóm.

8.2.6. Phát hiện và xử lý các giá trị ngoại lai

Việc lọc hoặc chuyển đổi các giá trị ngoại lai (outliers) phần lớn là được thao tác bằng các phép toán mảng. Hãy xem xét một DataFrame có một số dữ liệu phân phối chuẩn:

```
data = pd.DataFrame(np.random.standard_normal((1000, 4)))
data.describe()
# Out[96]:
#           0           1           2           3
# count  1000.000000  1000.000000  1000.000000  1000.000000
# mean    0.049091    0.026112   -0.002544   -0.051827
# std     0.996947    1.007458    0.995232    0.998311
# min    -3.645860   -3.184377   -3.745356   -3.428254
# 25%    -0.599807   -0.612162   -0.687373   -0.747478
# 50%     0.047101   -0.013609   -0.022158   -0.088274
# 75%     0.756646    0.695298    0.699046    0.623331
# max     2.653656    3.525865    2.735527    3.366626
```

	0	1	2	3
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	0.049091	0.026112	-0.002544	-0.051827
std	0.996947	1.007458	0.995232	0.998311
min	-3.645860	-3.184377	-3.745356	-3.428254
25%	-0.599807	-0.612162	-0.687373	-0.747478
50%	0.047101	-0.013609	-0.022158	-0.088274
75%	0.756646	0.695298	0.699046	0.623331
max	2.653656	3.525865	2.735527	3.366626

Giả sử bạn muốn tìm các giá trị trong một trong các cột vượt quá 3 về giá trị tuyệt đối:

```
col = data[2]
col[col.abs() > 3]
# Out[98]:
# 41    -3.399312
# 136   -3.745356
# Name: 2, dtype: float64
```

```
41      -3.399312
136     -3.745356
Name: 2, dtype: float64
```

Để chọn tất cả các hàng có một giá trị vượt quá 3 hoặc -3, bạn có thể sử dụng phương thức `any` trên một DataFrame chứa toàn các giá trị logical:

```
data[(data.abs() > 3).any(axis="columns")]
# Out[99]:
#           0          1          2          3
# 41  0.457246 -0.025907 -3.399312 -0.974657
# 60  1.951312  3.260383  0.963301  1.201206
# 136 0.508391 -0.196713 -3.745356 -1.520113
# 235 -0.242459 -3.056990  1.918403 -0.578828
# 258  0.682841  0.326045  0.425384 -3.428254
# 322  1.179227 -3.184377  1.369891 -1.074833
# 544 -3.548824  1.553205 -2.186301  1.277104
# 635 -0.578093  0.193299  1.397822  3.366626
# 782 -0.207434  3.525865  0.283070  0.544635
# 803 -3.645860  0.255475 -0.549574 -1.907459
```

	0	1	2	3
41	0.457246	-0.025907	-3.399312	-0.974657
60	1.951312	3.260383	0.963301	1.201206
136	0.508391	-0.196713	-3.745356	-1.520113
235	-0.242459	-3.056990	1.918403	-0.578828
258	0.682841	0.326045	0.425384	-3.428254
322	1.179227	-3.184377	1.369891	-1.074833
544	-3.548824	1.553205	-2.186301	1.277104
635	-0.578093	0.193299	1.397822	3.366626
782	-0.207434	3.525865	0.283070	0.544635
803	-3.645860	0.255475	-0.549574	-1.907459

Dấu ngoặc đơn xung quanh `data.abs() > 3` là cần thiết để gọi phương thức `any` trên kết quả của phép toán so sánh.

Các giá trị có thể được đặt dựa trên các tiêu chí này. Đoạn câu lệnh dưới đây là để giới hạn các giá trị nằm ngoài khoảng từ -3 đến 3:

```
data[data.abs() > 3] = np.sign(data) * 3
data.describe()
# Out[101]:
#           0           1           2           3
# count  1000.000000  1000.000000  1000.000000  1000.000000
# mean    0.050286    0.025567   -0.001399   -0.051765
# std     0.992920    1.004214    0.991414    0.995761
# min    -3.000000   -3.000000   -3.000000   -3.000000
# 25%    -0.599807   -0.612162   -0.687373   -0.747478
# 50%     0.047101   -0.013609   -0.022158   -0.088274
# 75%     0.756646    0.695298    0.699046    0.623331
# max     2.653656    3.000000    2.735527    3.000000
```

	0	1	2	3
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	0.050286	0.025567	-0.001399	-0.051765
std	0.992920	1.004214	0.991414	0.995761
min	-3.000000	-3.000000	-3.000000	-3.000000
25%	-0.599807	-0.612162	-0.687373	-0.747478
50%	0.047101	-0.013609	-0.022158	-0.088274
75%	0.756646	0.695298	0.699046	0.623331
max	2.653656	3.000000	2.735527	3.000000

Câu lệnh `np.sign(data)` tạo ra các giá trị 1 và -1 dựa trên việc các giá trị trong `data` là dương hay âm:

```
np.sign(data).head()
# Out[102]:
#           0           1           2           3
# 0  -1.0    1.0   -1.0    1.0
# 1   1.0   -1.0    1.0   -1.0
# 2   1.0    1.0    1.0   -1.0
# 3  -1.0   -1.0    1.0   -1.0
# 4  -1.0    1.0   -1.0   -1.0
```

	0	1	2	3
0	-1.0	1.0	-1.0	1.0
1	1.0	-1.0	1.0	-1.0
2	1.0	1.0	1.0	-1.0
3	-1.0	-1.0	1.0	-1.0
4	-1.0	1.0	-1.0	-1.0

8.2.7. Tính toán biến chỉ báo

Một kiểu biến đổi biến thường được sử dụng cho mục đích xây dựng mô hình thống kê hoặc các ứng dụng học máy là chuyển đổi một biến phân loại thành một ma trận biến giả hay biến chỉ báo. Nếu một cột trong DataFrame có J giá trị riêng biệt, bạn sẽ tạo ra một ma trận hoặc DataFrame mới có J cột chỉ chứa giá trị 0 và 1. Trong thư viện `pandas` có hàm

`pandas.get_dummies` để thực hiện điều này:

```
df = pd.DataFrame({"key": ["b", "b", "a", "c", "a", "b"],
                   "data1": range(6)})
```

```
df
```

```
# Out[116]:
```

```
#   key  data1
# 0   b      0
# 1   b      1
# 2   a      2
# 3   c      3
# 4   a      4
# 5   b      5
```

```
pd.get_dummies(df["key"], dtype=float)
```

```
# Out[117]:
```

```
#      a      b      c
# 0  0.0  1.0  0.0
# 1  0.0  1.0  0.0
# 2  1.0  0.0  0.0
# 3  0.0  0.0  1.0
# 4  1.0  0.0  0.0
# 5  0.0  1.0  0.0
```

	a	b	c
0	0.0	1.0	0.0
1	0.0	1.0	0.0
2	1.0	0.0	0.0
3	0.0	0.0	1.0
4	1.0	0.0	0.0
5	0.0	1.0	0.0

Trong hàm `get_dummies` đã sử dụng tham số `dtype=float` để thay đổi kiểu đầu ra sang số thực.

Trong một số trường hợp, bạn có thể muốn thêm một tiền tố vào tên các cột trong DataFrame.

`pandas.get_dummies` có tham số `prefix` để thực hiện việc này:

```
dummies = pd.get_dummies(df["key"], prefix="key", dtype=float)
df_with_dummy = df[["data1"]].join(dummies)
df_with_dummy
# Out[120]:
#   data1  key_a  key_b  key_c
# 0      0   0.0   1.0   0.0
# 1      1   0.0   1.0   0.0
# 2      2   1.0   0.0   0.0
# 3      3   0.0   0.0   1.0
# 4      4   1.0   0.0   0.0
# 5      5   0.0   1.0   0.0
```

	data1	key_a	key_b	key_c
0	0	0.0	1.0	0.0
1	1	0.0	1.0	0.0
2	2	1.0	0.0	0.0
3	3	0.0	0.0	1.0
4	4	1.0	0.0	0.0
5	5	0.0	1.0	0.0

Phương thức `join` sử dụng ở trên sẽ được giải thích chi tiết hơn trong chương tiếp theo.

Nếu một hàng trong `DataFrame` thuộc về nhiều danh mục, chúng ta phải sử dụng một cách tiếp cận khác để tạo các biến giả. Hãy xem ví dụ về dữ liệu dưới đây:

```
mnames = ["movie_id", "title", "genres"]
movies = pd.read_table("datasets/movielens/movies.dat", sep="::",
                      header=None, names=mnames, engine="python")

movies[:10]
# Out[123]:
```

	movie_id	title	genres
# 0	1	Toy Story (1995)	Animation Children's Comedy
# 1	2	Jumanji (1995)	Adventure Children's Fantasy
# 2	3	Grumpier Old Men (1995)	Comedy Romance
# 3	4	Waiting to Exhale (1995)	Comedy Drama
# 4	5	Father of the Bride Part II (1995)	Comedy
# 5	6	Heat (1995)	Action Crime Thriller
# 6	7	Sabrina (1995)	Comedy Romance
# 7	8	Tom and Huck (1995)	Adventure Children's
# 8	9	Sudden Death (1995)	Action
# 9	10	GoldenEye (1995)	Action Adventure Thriller


```

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[49], line 2
      1 mnames = ["movie_id", "title", "genres"]
----> 2 movies = pd.read_table("datasets/movielens/movies.dat", sep="::",
      3                       header=None, names=mnames, engine="python")
      4 movies[:10]
      5 # Out[123]:
      6 #      movie_id                                title                                genres
      7 # 0              1                    Toy Story (1995)    Animation|Children's|Comedy
(...)    15 # 8              9                    Sudden Death (1995)
      16 # 9              10                GoldenEye (1995)    Action|Adventure|Thriller

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\io\parsers\read
1392 kwds_defaults = _refine_defaults_read(
1393     dialect,
1394     delimiter,
(...)    1401     dtype_backend=dtype_backend,
1402 )
1403 kwds.update(kwds_defaults)
-> 1405 return _read(filepath_or_buffer, kwds)

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\io\parsers\read
617 _validate_names(kwds.get("names", None))
619 # Create the parser.
--> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
622 if chunksize or iterator:
623     return parser

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\io\parsers\read
1617     self.options["has_index_names"] = kwds["has_index_names"]
1619 self.handles: IOHandles | None = None
-> 1620 self._engine = self._make_engine(f, self.engine)

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\io\parsers\read
1878     if "b" not in mode:
1879         mode += "b"
-> 1880 self.handles = get_handle(
1881     f,
1882     mode,
1883     encoding=self.options.get("encoding", None),
1884     compression=self.options.get("compression", None),
1885     memory_map=self.options.get("memory_map", False),
1886     is_text=is_text,
1887     errors=self.options.get("encoding_errors", "strict"),
1888     storage_options=self.options.get("storage_options", None),
1889 )
1890 assert self.handles is not None
1891 f = self.handles.handle

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\io\common.py:87
868 elif isinstance(handle, str):
869     # Check whether the filename is to be opened in binary mode.
870     # Binary mode does not support 'encoding' and 'newline'.
871     if ioargs.encoding and "b" not in ioargs.mode:

```

```

872         # Encoding
--> 873         handle = open(
874             handle,
875             ioargs.mode,
876             encoding=ioargs.encoding,
877             errors=errors,
878             newline="",
879         )
880     else:
881         # Binary mode
882         handle = open(handle, ioargs.mode)

```

`FileNotFoundError: [Errno 2] No such file or directory: 'datasets/movielens/movies.dat'`

`pandas` có phương thức dành cho chuỗi ký tự `str.get_dummies` có thể xử lý trường hợp này:

```

dummies = movies["genres"].str.get_dummies("|")
dummies.iloc[:10, :6]
# Out[125]:
#   Action  Adventure  Animation  Children's  Comedy  Crime
# 0       0         0         1         1         1       0
# 1       0         1         0         1         0       0
# 2       0         0         0         0         1       0
# 3       0         0         0         0         1       0
# 4       0         0         0         0         1       0
# 5       1         0         0         0         0       1
# 6       0         0         0         0         1       0
# 7       0         1         0         1         0       0
# 8       1         0         0         0         0       0
# 9       1         1         0         0         0       0

```

Lưu ý: Đối với dữ liệu lớn hơn, phương pháp xây dựng các biến phân loại nhận nhiều giá trị sẽ không hiệu quả về thời gian. Sẽ tốt hơn nếu viết chúng ta xử lý dưới dạng mảng trong NumPy, sau đó chuyển sang kiểu DataFrame.

Một công thức hữu ích cho các ứng dụng thống kê là kết hợp `pandas.get_dummies` với một hàm rời rạc hóa như `pandas.cut`:

```

np.random.seed(12345) # để làm cho ví dụ có thể lặp lại
values = np.random.uniform(size=10)
values
# Out[130]: array([0.9296, 0.3164, 0.1839, 0.2046, 0.5677, 0.5955, 0.9645, 0.6532,
#                0.7489, 0.6536])

bins = [0, 0.2, 0.4, 0.6, 0.8, 1]
pd.get_dummies(pd.cut(values, bins))
# Out[132]:
#   (0.0, 0.2] (0.2, 0.4] (0.4, 0.6] (0.6, 0.8] (0.8, 1.0]
# 0      False      False      False      False      True
# 1      False       True      False      False      False
# 2       True      False      False      False      False
# 3      False       True      False      False      False
# 4      False      False       True      False      False
# 5      False      False       True      False      False
# 6      False      False      False      False      True
# 7      False      False      False      True      False
# 8      False      False      False      True      False
# 9      False      False      False      True      False

```

Chúng ta sẽ xem lại `pandas.get_dummies` sau các phần tiếp theo của sách.

8.3. Thao tác trên chuỗi ký tự

Python từ lâu đã là một ngôn ngữ phổ biến để thao tác dữ liệu thô một phần nhờ vào sự dễ dàng trong việc xử lý chuỗi và văn bản. Hầu hết các thao tác văn bản đều khả thi với các phương thức tích hợp sẵn của đối tượng chuỗi. Đối với các thao tác xử lý văn bản phức tạp hơn, có thể cần đến các **biểu thức chính quy**. `pandas` bổ sung vào đó bằng cách cho phép bạn áp dụng các biểu thức chuỗi và biểu thức chính quy một cách ngắn gọn trên toàn bộ mảng dữ liệu, đồng thời xử lý sự phiền phức của dữ liệu bị thiếu.

8.3.1. Các phương thức đối tượng chuỗi ký tự tích hợp sẵn của Python

Trong nhiều ứng dụng xử lý chuỗi và viết kịch bản, các phương thức chuỗi tích hợp sẵn là đủ. Ví dụ, một chuỗi được phân tách bằng dấu phẩy có thể được chia thành các phần bằng `split`:

```

val = "a,b, guido"
val.split(",")
# Out[152]: ['a', 'b', ' guido']

```

`split` thường được kết hợp với `strip` để loại bỏ khoảng trắng, bao gồm cả ngắt dòng:

```
pieces = [x.strip() for x in val.split(",")]
pieces
# Out[154]: ['a', 'b', 'guido']
```

Các chuỗi ký tự con này có thể được nối lại với nhau bằng một dấu hai chấm kép làm dấu phân cách bằng cách sử dụng phép cộng:

```
first, second, third = pieces
first + "::" + second + "::" + third
# Out[156]: 'a::b::guido'
```

Nhưng đây không phải là một phương pháp chung thực tế. Một cách nhanh hơn là sử dụng phương thức `join` trên chuỗi `"::"`:

```
"::".join(pieces)
# Out[157]: 'a::b::guido'
```

Có các phương thức khác liên quan đến việc định vị chuỗi con. Sử dụng từ khóa `in` của Python là cách tốt nhất để phát hiện một chuỗi con, mặc dù `index` và `find` cũng có thể được sử dụng:

```
"guido" in val
# Out[158]: True

val.index(",")
# Out[159]: 1

val.find(":")
# Out[160]: -1
```

Lưu ý rằng sự khác biệt giữa `find` và `index` là `index` sẽ báo lỗi nếu chuỗi ký tự không được tìm thấy thay vì trả về -1:

```
# val.index(":")
# -----
# ValueError                                Traceback (most recent call last)
# <ipython-input-161-bea4c4c30248> in <module>
# ----> 1 val.index(":")
#
# ValueError: substring not found
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-161-bea4c4c30248> in <module>  
----> 1 val.index(":")  
  
ValueError: substring not found
```

Liên quan đến điều này, `count` trả về số lần xuất hiện của một chuỗi con cụ thể:

```
val.count(",")  
# Out[162]: 2
```

Phương thức `replace` sẽ thay thế các lần xuất hiện của một chuỗi ký tự bằng một chuỗi ký tự khác. Phương thức này cũng thường được sử dụng để xóa các chuỗi ký tự bằng cách truyền một chuỗi ký tự trống:

```
val.replace(", ", " :")  
# Out[163]: 'a::b:: guido'  
  
val.replace(", ", "")  
# Out[164]: 'ab guido'
```

Xem Bảng liệt kê danh sách các phương thức chuỗi của Python.

Bảng ... các phương thức chuỗi tích hợp sẵn của Python

Phương thức	Mô tả
<code>count</code>	Trả về số lần xuất hiện không chồng chéo của chuỗi con trong chuỗi.
<code>endswith</code>	Trả về <code>True</code> nếu chuỗi kết thúc bằng hậu tố.
<code>startswith</code>	Trả về <code>True</code> nếu chuỗi bắt đầu bằng tiền tố.
<code>join</code>	Sử dụng chuỗi làm dấu phân cách để nối một chuỗi các chuỗi khác.
<code>index</code>	Trả về chỉ mục bắt đầu của lần xuất hiện đầu tiên của chuỗi con được truyền nếu tìm thấy trong chuỗi; ngược lại, gây ra <code>ValueError</code> nếu không tìm thấy.
<code>find</code>	Trả về vị trí của ký tự đầu tiên của lần xuất hiện đầu tiên của chuỗi con trong chuỗi; giống như <code>index</code> , nhưng trả về <code>-1</code> nếu không tìm thấy.
<code>rfind</code>	Trả về vị trí của ký tự đầu tiên của lần xuất hiện cuối cùng của chuỗi con trong chuỗi; trả về <code>-1</code> nếu không tìm thấy.
<code>replace</code>	Thay thế các lần xuất hiện của chuỗi bằng một chuỗi khác.
<code>strip</code> , <code>rstrip</code> , <code>lstrip</code>	Loại bỏ khoảng trắng, bao gồm cả ngắt dòng ở cả hai bên, ở bên phải, hoặc ở bên trái, tương ứng.
<code>split</code>	Chia chuỗi thành danh sách các chuỗi con bằng cách sử dụng dấu phân cách được truyền.
<code>lower</code>	Chuyển đổi các ký tự chữ cái thành chữ thường.
<code>upper</code>	Chuyển đổi các ký tự chữ cái thành chữ hoa.
<code>casefold</code>	Chuyển đổi các ký tự thành chữ thường, và chuyển đổi bất kỳ tổ hợp ký tự biến thể theo vùng cụ thể nào thành một dạng chung có thể so sánh được.
<code>ljust</code> , <code>rjust</code>	Căn lề trái hoặc căn lề phải, tương ứng; đệm phía đối diện của chuỗi bằng khoảng trắng (hoặc một ký tự điền khác) để trả về một chuỗi có chiều rộng tối thiểu.

8.3.2. Biểu thức chính quy

Biểu thức chính quy cung cấp một cách linh hoạt để tìm kiếm hoặc mapping các chuỗi ký tự trong văn bản. Một biểu thức, thường được gọi là *regex*, là một chuỗi được hình thành theo ngôn ngữ biểu thức chính quy. Thư viện `re` tích hợp sẵn của Python cho phép áp dụng các biểu thức chính quy cho các chuỗi; chúng ta sẽ tìm hiểu về *regex* thông qua một số ví dụ về cách sử dụng.

Lưu ý: Trình bày đầy đủ về biểu thức chính quy có thể yêu cầu một cuốn sách riêng và do đó nằm ngoài phạm vi của cuốn sách này. Có rất nhiều hướng dẫn và tài liệu tham khảo tuyệt vời có sẵn trên internet và trong các cuốn sách khác.

Các hàm của thư viện `re` được chia thành ba loại: mapping, thay thế và chia tách chuỗi ký tự. Hãy xem một ví dụ đơn giản: giả sử chúng ta muốn chia một chuỗi được phân tách bằng một số lượng ký tự khoảng trắng, có thể là tab, dấu cách hoặc ngắt dòng.

Regex mô tả một hoặc nhiều ký tự khoảng trắng là `\s+`:

```
import re
text = "foo    bar\t baz  \tqux"
re.split(r"\s+", text)
# Out[167]: ['foo', 'bar', 'baz', 'qux']
```

Khi bạn gọi `re.split(r"\s+", text)`, biểu thức chính quy được biên dịch trước, và sau đó phương thức `split` được gọi trên văn bản được truyền vào. Trong Python, thêm chữ `r` trước một chuỗi (ví dụ: `r"\s+"`) nghĩa là bạn đang viết raw string — chuỗi “thô”, để tránh việc phải sử dụng `\\`.

Bạn có thể tự tạo một đối tượng regex có thể tái sử dụng bằng phương thức `compile` như sau

```
regex = re.compile(r"\s+")
regex.split(text)
# Out[169]: ['foo', 'bar', 'baz', 'qux']
```

Nếu bạn muốn lấy một danh sách tất cả các mẫu khớp với regex, bạn có thể sử dụng phương thức `findall`:


```
regex.findall(text)
# Out[170]: [' ', '\t ', ' \t']
```

Việc tạo một đối tượng regex bằng `re.compile` được khuyến khích nếu bạn có ý định áp dụng cùng một biểu thức cho nhiều chuỗi; làm như vậy sẽ tiết kiệm thời gian tính toán thay vì phải gọi thư viện `re` cho từng lần.

`match` và `search` có liên quan chặt chẽ với `findall`. Trong khi `findall` trả về tất cả các kết quả khớp trong một chuỗi, `search` chỉ trả về kết quả khớp đầu tiên. Chặt chẽ hơn, `match` chỉ khớp ở đầu chuỗi. Hãy xem xét một văn bản và một biểu thức chính quy có khả năng xác định hầu hết các địa chỉ email trong một văn bản:

```
text = """Dave dave@google.com
Steve steve@gmail.com
Rob rob@gmail.com
Ryan ryan@yahoo.com"""
pattern = r"[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,4}"

# re.IGNORECASE làm cho regex không phân biệt chữ hoa chữ thường
regex = re.compile(pattern, flags=re.IGNORECASE)
```

Sử dụng `findall` trên văn bản sẽ tạo ra một danh sách các địa chỉ email:

```
regex.findall(text)
# Out[172]: ['dave@google.com', 'steve@gmail.com', 'rob@gmail.com', 'ryan@yahoo.com']
```

`search` trả về một đối tượng khớp đặc biệt cho địa chỉ email đầu tiên trong văn bản. Đối với regex ở trên, đối tượng khớp chỉ có thể cho chúng ta biết vị trí bắt đầu và kết thúc của mẫu trong chuỗi:

```
m = regex.search(text)
m
# Out[174]: <re.Match object; span=(5, 20), match='dave@google.com'>

text[m.start():m.end()]
# Out[175]: 'dave@google.com'
```

`regex.match` trả về `None`, vì nó sẽ chỉ khớp nếu mẫu xuất hiện ở đầu chuỗi:

```
print(regex.match(text))  
# Out: None
```

None

Cũng liên quan đến tìm kiếm và khớp văn bản, `sub` sẽ trả về một chuỗi mới với các lần xuất hiện của mẫu được thay thế bằng một chuỗi mới:

```
print(regex.sub("REDACTED", text))  
# Out:  
# Dave REDACTED  
# Steve REDACTED  
# Rob REDACTED  
# Ryan REDACTED
```

Dave REDACTED
Steve REDACTED
Rob REDACTED
Ryan REDACTED

Giả sử bạn muốn tìm các địa chỉ email và đồng thời phân đoạn từng địa chỉ thành ba thành phần của nó: tên người dùng, tên miền và hậu tố tên miền. Để làm điều này, hãy đặt dấu ngoặc đơn xung quanh các phần của mẫu để phân đoạn:

```
pattern = r"([A-Z0-9._%+-]+)@([A-Z0-9.-]+\.[A-Z]{2,4})"  
regex = re.compile(pattern, flags=re.IGNORECASE)
```

Một đối tượng khớp được tạo bởi regex đã sửa đổi này trả về một tuple các thành phần mẫu bằng phương thức `groups` của nó:

```
m = regex.match("wesm@bright.net")  
m.groups()  
# Out[181]: ('wesm', 'bright.net', 'net') # Chú ý: trang web gốc có 'bright' thay vì '
```

`findall` trả về một danh sách các tuple khi mẫu có các nhóm:

```
regex.findall(text)
# Out[182]: [('dave', 'google.com', 'com'), ('steve', 'gmail.com', 'com'), ('rob', 'gm
```

`sub` cũng có quyền truy cập vào các nhóm trong mỗi kết quả khớp bằng cách sử dụng các ký hiệu đặc biệt như `\1` và `\2`. Ký hiệu `\1` tương ứng với nhóm khớp đầu tiên, `\2` tương ứng với nhóm thứ hai, v.v.:

```
print(regex.sub(r"Username: \1, Domain: \2, Suffix: \3", text))
# Out:
# Dave Username: dave, Domain: google.com, Suffix: com
# Steve Username: steve, Domain: gmail.com, Suffix: com
# Rob Username: rob, Domain: gmail.com, Suffix: com
# Ryan Username: ryan, Domain: yahoo.com, Suffix: com
```

```
Dave Username: dave, Domain: google.com, Suffix: com
Steve Username: steve, Domain: gmail.com, Suffix: com
Rob Username: rob, Domain: gmail.com, Suffix: com
Ryan Username: ryan, Domain: yahoo.com, Suffix: com
```

Có rất nhiều điều khác về biểu thức chính quy trong Python, hầu hết nằm ngoài phạm vi của cuốn sách này. Bảng 7.5 cung cấp một bản tóm tắt ngắn gọn.

Bảng ... Các phương thức biểu thức chính quy

Phương thức	Mô tả
<code>findall</code>	Trả về tất cả các mẫu khớp không chồng chéo trong một chuỗi dưới dạng một danh sách.
<code>finditer</code>	Giống như <code>findall</code> , nhưng trả về một iterator.
<code>match</code>	Khớp mẫu ở đầu chuỗi và tùy chọn phân đoạn các thành phần mẫu thành các nhóm; nếu mẫu khớp, trả về một đối tượng khớp, ngược lại là <code>None</code> .
<code>search</code>	Quét chuỗi để tìm khớp với mẫu, trả về một đối tượng <code>Match</code> nếu có; không giống như <code>match</code> , khớp có thể ở bất kỳ đâu trong chuỗi thay vì chỉ ở đầu.
<code>split</code>	Chia chuỗi thành các phần ở mỗi lần xuất hiện của mẫu.
<code>sub</code> , <code>subn</code>	Thay thế tất cả (<code>sub</code>) hoặc n lần xuất hiện đầu tiên (<code>subn</code>) của mẫu trong chuỗi bằng biểu thức thay thế; sử dụng các ký hiệu <code>\1</code> , <code>\2</code> , ... để tham chiếu đến các phần tử nhóm khớp trong chuỗi thay thế.

8.4. Dữ liệu kiểu phân loại

Phần này của chương giới thiệu về kiểu dữ liệu phân loại, hay còn gọi là `Categorical` của pandas. Trước hết, chúng ta thảo luận về cách có thể đạt được hiệu suất tốt hơn và sử dụng bộ nhớ ít hơn trong một số hoạt động pandas bằng cách sử dụng nó. Sau đó, chúng ta sẽ được làm quen với một số công cụ có thể giúp sử dụng dữ liệu phân loại trong phân tích dữ liệu và xây dựng mô hình.

8.4.1. Dữ liệu kiểu phân loại là gì?

Rất thường gặp trong thực tế dữ liệu là một cột trong bảng có thể chứa các giá trị lặp đi lặp lại của một tập nhỏ hơn các giá trị riêng biệt. Chúng ta đã thấy các hàm như `unique` và `value_counts` cho phép trích xuất các giá trị riêng biệt từ một mảng và tính toán tần suất của chúng, tương ứng:

```

values = pd.Series(['apple', 'orange', 'apple',
                    'apple'] * 2)

values
# Out[200]:
# 0      apple
# 1     orange
# 2      apple
# 3      apple
# 4      apple
# 5     orange
# 6      apple
# 7      apple
# dtype: object

pd.unique(values)
# Out[201]: array(['apple', 'orange'], dtype=object)

pd.value_counts(values)
# Out[202]:
# apple      6
# orange     2
# Name: count, dtype: int64

```

Nhiều hệ thống dữ liệu đã phát triển các phương pháp chuyên biệt để biểu diễn dữ liệu với các giá trị lặp lại nhằm lưu trữ và tính toán hiệu quả hơn. Một trong những phương pháp phổ biến là sử dụng các *dimension tables* chứa các giá trị riêng biệt và lưu trữ các quan sát chính dưới dạng các số nguyên tham chiếu đến bảng chiều:

```

values = pd.Series([0, 1, 0, 0] * 2)
dim = pd.Series(['apple', 'orange'])
values
# Out[205]:
# 0      0
# 1      1
# 2      0
# 3      0
# 4      0
# 5      1
# 6      0
# 7      0
# dtype: int64

dim
# Out[206]:
# 0      apple
# 1     orange
# dtype: object

```

Chúng ta có thể sử dụng phương thức `take` để khôi phục lại mảng ban đầu:

```
dim.take(values)
# Out[207]:
# 0      apple
# 1     orange
# 0      apple
# 0      apple
# 0      apple
# 1     orange
# 0      apple
# 0      apple
# dtype: object
```

Biểu diễn này dưới dạng số nguyên được gọi là biểu diễn *phân loại* hoặc *mã hóa từ điển* (*dictionary-encoded*). Mảng các giá trị riêng biệt có thể được gọi là các *phân loại* (*categories*), *từ điển* (*dictionary*), hoặc *cấp độ* (*levels*) của dữ liệu. Trong cuốn sách này, chúng ta sẽ sử dụng các thuật ngữ *phân loại* (*categories*). Các giá trị số nguyên tham chiếu đến các category được gọi là *mã phân loại* (*category codes*).

Biểu diễn phân loại có thể mang lại những cải thiện đáng kể về hiệu suất khi bạn thực hiện phân tích. Bạn cũng có thể thực hiện các phép biến đổi trên các phân loại trong khi vẫn giữ nguyên các câu lệnh. Một số ví dụ về các phép biến đổi có thể được thực hiện với chi phí tương đối thấp là:

- Đổi tên các phân loại
- Thêm một phân loại mới mà không thay đổi thứ tự hoặc vị trí của các phân loại hiện có

8.4.2. Kiểu phân loại trong pandas

pandas có một kiểu mở rộng **Categorical** đặc biệt để lưu trữ dữ liệu sử dụng biểu diễn hoặc mã hóa phân loại dựa trên số nguyên. Đây là một kỹ thuật nén dữ liệu phổ biến cho dữ liệu có nhiều lần xuất hiện của các giá trị tương tự và có thể cung cấp hiệu suất nhanh hơn đáng kể với việc sử dụng bộ nhớ thấp hơn, đặc biệt đối với dữ liệu chuỗi.

Hãy xem xét ví dụ Series từ trước:

```

fruits = ['apple', 'orange', 'apple', 'apple'] * 2
N = len(fruits)
rng = np.random.default_rng(seed=12345)
df = pd.DataFrame({'fruit': fruits,
                   'basket_id': np.arange(N),
                   'count': rng.integers(3, 15, size=N),
                   'weight': rng.uniform(0, 4, size=N)},
                  columns=['basket_id', 'fruit', 'count', 'weight'])

df
# Out[212]:
#   basket_id  fruit  count  weight
# 0         0  apple     11  1.564438
# 1         1 orange      5  1.331256
# 2         2  apple     12  2.393235
# 3         3  apple      6  0.746937
# 4         4  apple      5  2.691024
# 5         5 orange     12  3.767211
# 6         6  apple     10  0.992983
# 7         7  apple     11  3.795525

```

Ở đây, `df['fruit']` là một mảng các đối tượng chuỗi ký tự của Python. Chúng ta có thể chuyển đổi nó thành phân loại bằng cách gọi:

```

fruit_cat = df['fruit'].astype('category')
fruit_cat
# Out[214]:
# 0    apple
# 1   orange
# 2    apple
# 3    apple
# 4    apple
# 5   orange
# 6    apple
# 7    apple
# Name: fruit, dtype: category
# Categories (2, object): ['apple', 'orange']

```

Các giá trị cho `fruit_cat` bây giờ là một `pandas.Categorical` và có thể truy cập thông qua thuộc tính `.array`:

```

c = fruit_cat.array
type(c)
# Out[216]: pandas.core.arrays.categorical.Categorical

```

Đối tượng `Categorical` có các thuộc tính `categories` và `codes`:

```
c.categories
# Out[217]: Index(['apple', 'orange'], dtype='object')

c.codes
# Out[218]: array([0, 1, 0, 0, 0, 1, 0, 0], dtype=int8)
```

Một cách để lấy ánh xạ giữa mã phân loại và giá trị là:

```
dict(enumerate(c.categories))
# Out[219]: {0: 'apple', 1: 'orange'}
```

Bạn có thể chuyển đổi một cột DataFrame thành phân loại bằng cách gán kết quả đã chuyển đổi:

```
df['fruit'] = df['fruit'].astype('category')
df["fruit"]
# Out[221]:
# 0    apple
# 1   orange
# 2    apple
# 3    apple
# 4    apple
# 5   orange
# 6    apple
# 7    apple
# Name: fruit, dtype: category
# Categories (2, object): ['apple', 'orange']
```

Bạn cũng có thể tạo `pandas.Categorical` trực tiếp từ các loại chuỗi Python khác:

```
my_categories = pd.Categorical(['foo', 'bar', 'baz', 'foo', 'bar'])
my_categories
# Out[223]:
# ['foo', 'bar', 'baz', 'foo', 'bar']
# Categories (3, object): ['bar', 'baz', 'foo']
```

Nếu bạn đã thu được dữ liệu được mã hóa phân loại từ một nguồn khác, bạn có thể sử dụng hàm tạo `from_codes` thay thế:


```
categories = ['foo', 'bar', 'baz']
codes = [0, 1, 2, 0, 0, 1]
my_cats_2 = pd.Categorical.from_codes(codes, categories)
my_cats_2
# Out[227]:
# ['foo', 'bar', 'baz', 'foo', 'foo', 'bar']
# Categories (3, object): ['foo', 'bar', 'baz']
```

Trừ khi được chỉ định rõ ràng, các chuyển đổi phân loại giả định không có thứ tự cụ thể của các phân loại. Vì vậy, mảng các phân loại có thể theo một thứ tự khác nhau tùy thuộc vào thứ tự của dữ liệu đầu vào. Khi sử dụng `from_codes` hoặc bất kỳ hàm tạo nào khác, bạn có thể chỉ ra rằng các phân loại có một thứ tự có ý nghĩa:

```
ordered_cat = pd.Categorical.from_codes(codes, categories,
                                         ordered=True)

ordered_cat
# Out[229]:
# ['foo', 'bar', 'baz', 'foo', 'foo', 'bar']
# Categories (3, object): ['foo' < 'bar' < 'baz']
```

Đầu ra `[foo < bar < baz]` cho biết 'foo' đứng trước 'bar' trong thứ tự, v.v. Một thể hiện phân loại không có thứ tự có thể được sắp xếp thứ tự bằng `as_ordered`:

```
my_cats_2.as_ordered()
# Out[230]:
# ['foo', 'bar', 'baz', 'foo', 'foo', 'bar']
# Categories (3, object): ['foo' < 'bar' < 'baz']
```

Lưu ý cuối cùng, dữ liệu phân loại không nhất thiết phải là chuỗi, mặc dù chúng ta chỉ hiển thị các ví dụ về chuỗi. Một mảng phân loại có thể bao gồm bất kỳ kiểu giá trị bất biến nào.

8.4.3. Tính toán với dữ liệu phân loại

Việc sử dụng `Categorical` trong pandas so với phiên bản không mã hóa như một mảng các chuỗi thường hoạt động theo cùng một cách. Một số phần của `pandas`, như hàm `groupby`, hoạt động tốt hơn khi làm việc với dữ liệu phân loại. Cũng có một số hàm có thể sử dụng `ordered`.

Hãy xem xét một số dữ liệu số ngẫu nhiên và sử dụng hàm phân nhóm `pandas.qcut`. Hàm này trả về `pandas.Categorical`; chúng ta đã sử dụng `pandas.cut` trước đó trong sách nhưng đã bỏ

qua chi tiết về cách hoạt động của dữ liệu phân loại:

```
rng = np.random.default_rng(seed=12345)
draws = rng.standard_normal(1000)
draws[:5]
# Out[233]: array([-1.4238,  1.2637, -0.8707, -0.2592, -0.0753])
```

Hãy tính toán một phân nhóm tứ phân vị của dữ liệu này và trích xuất một số thống kê:

```
bins = pd.qcut(draws, 4)
bins
# Out[235]:
# [(-3.121, -0.675], (0.687, 3.211], (-3.121, -0.675], (-0.675, 0.0134], (-0.675, 0.01
# (0.0134, 0.687], (0.0134, 0.687], (-0.675, 0.0134], (0.0134, 0.687], (-0.675, 0.0134
# Length: 1000
# Categories (4, interval[float64, right]): [(-3.121, -0.675] < (-0.675, 0.0134] < (0.
```

Mặc dù hữu ích, các phân vị mẫu chính xác có thể ít hữu ích hơn để tạo báo cáo so với tên tứ phân vị. Chúng ta có thể đạt được điều này bằng đối số `labels` cho `qcut`:

```
bins = pd.qcut(draws, 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
bins
# Out[237]:
# ['Q1', 'Q4', 'Q1', 'Q2', 'Q2', ..., 'Q3', 'Q3', 'Q2', 'Q3', 'Q2']
# Length: 1000
# Categories (4, object): ['Q1' < 'Q2' < 'Q3' < 'Q4']

bins.codes[:10]
# Out[238]: array([0, 3, 0, 1, 1, 0, 0, 2, 2, 0], dtype=int8)
```

Dữ liệu phân loại `bins` được gắn nhãn không chứa thông tin về các cạnh nhóm trong dữ liệu, vì vậy chúng ta có thể sử dụng `groupby` để trích xuất một số thống kê tóm tắt:

```
bins = pd.Series(bins, name='quartile')
results = (pd.Series(draws)
            .groupby(bins)
            .agg(['count', 'min', 'max'])
            .reset_index())

results
# Out[241]:
#   quartile  count      min      max
# 0        Q1    250 -3.119609 -0.678494
# 1        Q2    250 -0.673305  0.008009
# 2        Q3    250  0.018753  0.686183
# 3        Q4    250  0.688282  3.211418
```

Cột `'quartile'` trong kết quả giữ lại thông tin phân loại ban đầu, bao gồm cả thứ tự, từ `bins`:

```
results['quartile']
# Out[242]:
# 0    Q1
# 1    Q2
# 2    Q3
# 3    Q4
# Name: quartile, dtype: category
# Categories (4, object): ['Q1' < 'Q2' < 'Q3' < 'Q4']
```

8.4.4. Đo lường hiệu suất khi sử dụng dữ liệu phân loại

Ở phần trước, chúng ta đã nói rằng các kiểu phân loại có thể cải thiện hiệu suất và việc sử dụng bộ nhớ, vì vậy hãy xem một số ví dụ. Hãy xem xét một số Series với 10 triệu phần tử và một số lượng nhỏ các phân loại riêng biệt:

```
N = 10_000_000
labels = pd.Series(['foo', 'bar', 'baz', 'qux'] * (N // 4))
```

Bây giờ chúng ta chuyển đổi `labels` thành phân loại:

```
categories = labels.astype('category')
```

Bây giờ chúng ta lưu ý rằng `labels` sử dụng bộ nhớ nhiều hơn đáng kể so với `categories`:

```
labels.memory_usage(deep=True)
# Out[246]: 600000128

categories.memory_usage(deep=True)
# Out[247]: 10000540
```

Tất nhiên, việc chuyển đổi sang `category` cũng sẽ cần thời gian tính toán, nhưng đó là tính toán một lần:

```
%time _ = labels.astype('category')
# CPU times: user 279 ms, sys: 6.06 ms, total: 285 ms
# Wall time: 285 ms
```

Các thao tác `GroupBy` có thể nhanh hơn đáng kể với dữ liệu phân loại vì các thuật toán cơ bản sử dụng mảng mã dựa trên số nguyên thay vì một mảng các chuỗi. Ở đây chúng ta so sánh hiệu suất của `value_counts()`, mà bên trong sử dụng cơ chế `GroupBy`:

```
%timeit labels.value_counts()
# 331 ms +- 5.39 ms per loop (mean +- std. dev. of 7 runs, 1 loop each)

%timeit categories.value_counts()
# 15.6 ms +- 152 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

8.4.5. Phương thức phân loại

Series chứa dữ liệu phân loại có một số phương thức đặc biệt tương tự như các phương thức chuỗi chuyên biệt `Series.str`. Điều này cũng cung cấp quyền truy cập thuận tiện vào các phân loại và mã. Hãy xem xét Series:

```
s = pd.Series(['a', 'b', 'c', 'd'] * 2)
cat_s = s.astype('category')
cat_s
# Out[253]:
# 0    a
# 1    b
# 2    c
# 3    d
# 4    a
# 5    b
# 6    c
# 7    d
# dtype: category
# Categories (4, object): ['a', 'b', 'c', 'd']
```

Thuộc tính truy cập đặc biệt `cat` cung cấp quyền truy cập vào các phương thức phân loại:

```
cat_s.cat.codes
# Out[254]:
# 0    0
# 1    1
# 2    2
# 3    3
# 4    0
# 5    1
# 6    2
# 7    3
# dtype: int8

cat_s.cat.categories
# Out[255]: Index(['a', 'b', 'c', 'd'], dtype='object')
```

Giả sử rằng chúng ta biết tập hợp các phân loại thực tế cho dữ liệu này mở rộng ra ngoài bốn giá trị được quan sát trong dữ liệu. Chúng ta có thể sử dụng phương thức `set_categories` để thay đổi chúng:

```
actual_categories = ['a', 'b', 'c', 'd', 'e']
cat_s2 = cat_s.cat.set_categories(actual_categories)
cat_s2
# Out[258]:
# 0    a
# 1    b
# 2    c
# 3    d
# 4    a
# 5    b
# 6    c
# 7    d
# dtype: category
# Categories (5, object): ['a', 'b', 'c', 'd', 'e']
```

Mặc dù có vẻ như dữ liệu không thay đổi, các phân loại mới sẽ được phản ánh trong các hoạt động sử dụng chúng. Ví dụ, `value_counts` giữ nguyên các phân loại, nếu có:

```
cat_s.value_counts()
# Out[259]:
# a    2
# b    2
# c    2
# d    2
# Name: count, dtype: int64

cat_s2.value_counts()
# Out[260]:
# a    2
# b    2
# c    2
# d    2
# e    0
# Name: count, dtype: int64
```

Trong các tập dữ liệu lớn, dữ liệu phân loại thường được sử dụng như một công cụ tiện lợi để tiết kiệm bộ nhớ và cải thiện hiệu suất. Sau khi bạn lọc một DataFrame hoặc Series lớn, nhiều phân loại có thể không xuất hiện trong dữ liệu. Để giải quyết vấn đề này, chúng ta có thể sử dụng phương thức `remove_unused_categories` để loại bỏ các phân loại không được quan sát:

```
cat_s3 = cat_s[cat_s.isin(['a', 'b'])]
cat_s3
# Out[262]:
# 0    a
# 1    b
# 4    a
# 5    b
# dtype: category
# Categories (4, object): ['a', 'b', 'c', 'd']

cat_s3.cat.remove_unused_categories()
# Out[263]:
# 0    a
# 1    b
# 4    a
# 5    b
# dtype: category
# Categories (2, object): ['a', 'b']
```

Xem Bảng ... để biết danh sách các phương thức phân loại có sẵn.

Bảng ...: Các phương thức phân loại cho Series trong pandas

Phương thức	Mô tả
<code>add_categories</code>	Thêm các phân loại mới (chưa sử dụng) vào cuối các phân loại hiện có.
<code>as_ordered</code>	Làm cho các phân loại có thứ tự.
<code>as_unordered</code>	Làm cho các phân loại không có thứ tự.
<code>remove_categories</code>	Loại bỏ các phân loại, đặt bất kỳ giá trị bị loại bỏ nào thành null.
<code>remove_unused_categories</code>	Loại bỏ bất kỳ giá trị phân loại nào không xuất hiện trong dữ liệu.
<code>rename_categories</code>	Thay thế các phân loại bằng tập hợp các tên phân loại mới được chỉ định; không thể thay đổi số lượng phân loại.
<code>reorder_categories</code>	Hoạt động giống như <code>rename_categories</code> , nhưng cũng có thể thay đổi kết quả để có các phân loại có thứ tự.
<code>set_categories</code>	Thay thế các phân loại bằng tập hợp các phân loại mới được chỉ định; có thể thêm hoặc loại bỏ các phân loại.

8.4.5.1. Tạo biến giả để lập mô hình

Khi bạn sử dụng các công cụ thống kê hoặc học máy, bạn sẽ thường xuyên chuyển đổi dữ liệu phân loại thành các *biến giả* (*dummy variables*), còn được gọi là *mã hóa one-hot* (*one-hot encoding*). Điều này liên quan đến việc tạo một DataFrame với một cột cho mỗi phân loại riêng biệt; các cột này chứa 1 cho các lần xuất hiện của một phân loại nhất định và 0 cho các trường hợp khác.

Hãy xem xét ví dụ trước:

```
cat_s = pd.Series(['a', 'b', 'c', 'd'] * 2, dtype='category')
```

Như đã đề cập trước đó trong chương này, hàm `pandas.get_dummies` chuyển đổi dữ liệu phân loại một chiều này thành một DataFrame chứa biến giả:


```
pd.get_dummies(cat_s, dtype=float)
# Out[265]:
#      a      b      c      d
# 0  1.0  0.0  0.0  0.0
# 1  0.0  1.0  0.0  0.0
# 2  0.0  0.0  1.0  0.0
# 3  0.0  0.0  0.0  1.0
# 4  1.0  0.0  0.0  0.0
# 5  0.0  1.0  0.0  0.0
# 6  0.0  0.0  1.0  0.0
# 7  0.0  0.0  0.0  1.0
```

Như vậy, bạn đọc có thể thấy rằng việc chuẩn bị dữ liệu hiệu quả có thể cải thiện đáng kể năng suất bằng cách cho phép bạn dành nhiều thời gian hơn để phân tích dữ liệu và ít thời gian hơn để chuẩn bị dữ liệu cho phân tích. Chúng ta đã khám phá một số công cụ trong chương này, nhưng phạm vi bao phủ ở đây không hề toàn diện. Trong chương tiếp theo, chúng ta sẽ khám phá chức năng thêm các chức năng quan trọng khác của `pandas` trong phân tích dữ liệu.